

Computational Thinking with Python

Session 1 – Python Basics

TODAY WE COVER OUTPUT, VARIABLES, INPUT, AND MATH

At the end of this session, you should be able to comprehend, trace, adapt and create Python code that:

- **Outputs** text to the console
- Assigns data to **variables**
- Gets **input** and assigns it to variables
- Refers to variables in other statements
- Performs simple **maths** (addition, subtraction, multiplication, division)

AGENDA

- 1 Output
- 2 Variables
- 3 Input
- 4 Math

AGENDA

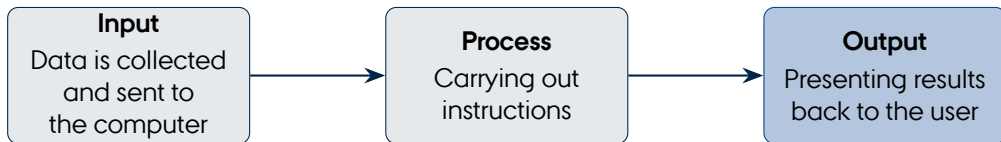
1 Output

2 Variables

3 Input

4 Math

PROGRAMS FOLLOW THE INPUT-PROCESS-OUTPUT PATTERN



THE PRINT() FUNCTION DISPLAYS TEXT ON THE SCREEN

- The `print()` function displays a message on the screen
- You can print text (strings), numbers, or variables

Output

```
Hello, World!  
42  
3.14
```

Examples

```
print("Hello, World!") # text  
print(42) # integer  
print(3.14) # decimal
```

PYTHON IS CASE-SENSITIVE AND REQUIRES CORRECT PUNCTUATION

```
print("Hello, World!")
```

- `print` – lowercase p (Python is case-sensitive!)
- **Parentheses** – after `print` and at the end
- **Quotes** – text must be in quotes (`"..."` or `'...'`)

Common Mistakes

```
Print("Hello") # Error! Capital P  
print("Hello) # Error! Missing quote
```

APPLY PRINT() BY PREDICTING AND MODIFYING CODE

Practice: Open 01_Output in your Codespace

Complete: 01_predict.py, 02_predict.py, 03_investigate.py, 04_investigate.py,
05_investigate.py, 06_modify.py, 07_make.py

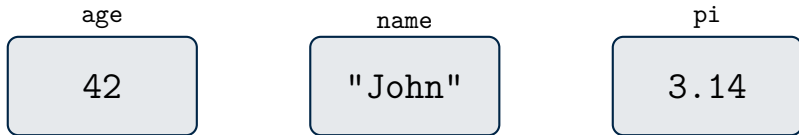
Predict → Investigate → Modify → Make

AGENDA

- 1 Output
- 2 Variables**
- 3 Input
- 4 Math

VARIABLES ARE LABELED CONTAINERS THAT STORE DATA

- A variable is like a **labeled box** that stores a value
- The **label** is the variable name, the **contents** are the value
- You can store different types: **numbers**, **decimals**, or **text**
- Variables let you save data to use later in your program



THE EQUAL SIGN ASSIGNS VALUES TO VARIABLES

- Use the **equal sign (=)** for assignment
- The **variable name** goes on the **left**
- The **value** goes on the **right**

name = value

Example

```
counter = 100 # integer
miles = 1000.0 # float
name = "John" # string

print(counter)
print(miles)
print(name)
```

Output

```
100 1000.0 John
```

PYTHON DISTINGUISHES BETWEEN NUMBERS AND TEXT

Data stored in variables can be of different **types**:

Numbers

- **int** (integers) – whole numbers: 10, -5, 1000
- **float** (floating point) – decimals: 3.14, -0.5, 99.99

Text

- **str** (strings) – text in quotes: "Hello", 'World'

The data type determines what operations you can perform!

F-STRINGS EMBED VARIABLES INSIDE TEXT

- Prefix string with `f`
- Put variables in **curly braces** `{...}`
- Much cleaner than concatenation

Output

```
Hi! My name is Homer Simpson  
I am 39 years old.
```

Code

```
name = "Homer Simpson"  
age = 39  
print(f"Hi! My name is {name}")  
print(f"I am {age} years old.")
```

Use f-strings for clean output!

STRING SLICING EXTRACTS PARTS OF TEXT

- Syntax: `string[start:end]` – extracts from start to end-1
- **Index** starts at **0**; negative indices count from the end

```
text = "Hello, World!"  
print(text[0:5]) # Hello  
print(text[-1]) # ! (last character)
```

H	e	l	l	o	,		W	o	r	l	d	!
0	1	2	3	4	5	6	7	8	9	10	11	12

SOLIDIFY VARIABLE CONCEPTS THROUGH HANDS-ON CODING

Practice: Open 02_Variables in your Codespace

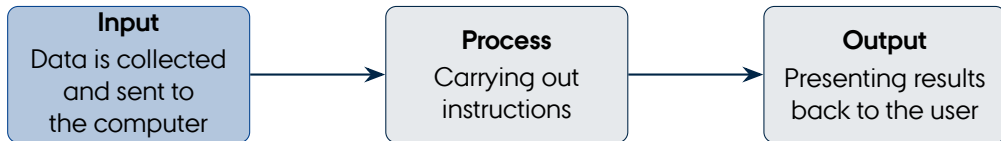
Complete: 01_predict.py, 02_predict.py, 03_investigate.py, 04_investigate.py,
05_modify.py

Predict → Investigate → Modify

AGENDA

- 1 Output
- 2 Variables
- 3 Input**
- 4 Math

INPUT COLLECTS DATA FROM THE USER



THE INPUT() FUNCTION ASKS THE USER FOR DATA

- `input()` pauses and waits for user input
- Store the result in a **variable**
- **Important:** `input()` always returns a string!

Code

```
name = input("What is your name?  
")  
print(f"Hello, {name}!")
```

Console

```
What is your name? Alice  
Hello, Alice!
```

The highlighted text shows what the user typed.

PRACTICE CAPTURING USER INPUT IN REAL PROGRAMS

Practice: Open 03_Input in your Codespace

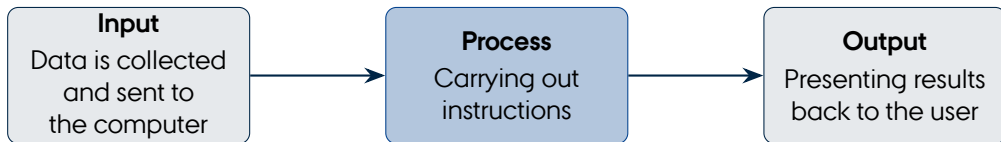
Complete: 01_predict.py, 02_predict.py, 03_investigate.py, 04_investigate.py,
05_modify.py

Predict → Investigate → Modify

AGENDA

- 1 Output
- 2 Variables
- 3 Input
- 4 **Math**

PROCESSING TRANSFORMS INPUT DATA INTO RESULTS



PYTHON SUPPORTS STANDARD ARITHMETIC OPERATORS

Operator	Name	Example
+	Addition	$x + y$
-	Subtraction	$x - y$
*	Multiplication	$x * y$
/	Division	x / y
%	Modulus (remainder)	$x \% y$
**	Exponentiation	$x ** y$
//	Floor division	$x // y$

VARIABLES STORE AND REUSE CALCULATION RESULTS

Workflow

1. Assign numbers to variables
2. Perform calculation
3. Store result in new variable
4. Output the result

Code

```
num1 = 5  
num2 = 10  
result = num1 + num2  
print(result)
```

Output

15

SHORTHAND OPERATORS SIMPLIFY VARIABLE UPDATES

```
score = 0
score += 1 # Same as: score = score + 1
print(score) # Output: 1
```

Shorthand Operators

<code>x += 5</code>	shorthand for <code>x = x + 5</code>
<code>x -= 5</code>	shorthand for <code>x = x - 5</code>
<code>x *= 5</code>	shorthand for <code>x = x * 5</code>
<code>x /= 5</code>	shorthand for <code>x = x / 5</code>

CONVERT STRINGS TO NUMBERS BEFORE MATH

The Problem

- `input()` returns a **string**
- Math with strings fails!

"5" + "3" # Result: "53" (oops!)

The Solution

```
num = int(input("Number: "))  
price = float(input("Price: "))
```

5 + 3 # Result: 8 (correct!)

Key Takeaway: Use `int()` for whole numbers, `float()` for decimals.

BUILD CALCULATION SKILLS WITH GUIDED EXERCISES

Practice: Open 04_Math in your Codespace

Complete: 01_predict.py, 02_predict.py, 03_investigate.py, 04_investigate.py,
05_modify.py

Predict → Investigate → Modify

COMPLETE THE SESSION WITH MODIFY & MAKE EXERCISES

Practice: Open 05_Modify_Make in your Codespace

Complete: 01_modify.py, 02_modify.py, 03_make.py, 04_make.py

Modify existing code → Create new programs from scratch

YOU NOW HAVE THE BUILDING BLOCKS FOR SIMPLE PYTHON PROGRAMS

- ✓ **Output** text to the console using `print()`
- ✓ **Variables** store data with meaningful names
 - Data types: `int`, `float`, `str`
 - Use f-strings for formatted output
- ✓ **Input** from user with `input()` – always returns a string
- ✓ **Math** operations: `+`, `-`, `*`, `/`, `//`, `%`, `**`
 - Convert strings to numbers with `int()` or `float()`

Thank You!

Nikolai Stein

